ertain period of time.

- Weight: Apply a weight value to your issue to measure the time, complexity, or value a given issue has or costs.

- Dates: Use in issues to keep track of deadlines and make sure features are shipped on time.

- Health status: You can associate one of four predefined health status labels to your issue: on track, needs attention, at risk, or needs review.

- Parent: Connects an issue to an epic.

- Time Tracking: Use time tracking to estimate and measure your team·s work on an issue, or how much work is expected to be done to complete the issue.

- Contacts: Attach contacts to issues for additional collaboration, meeting follow-ups, or progress updates. This also adds the contact's avatar next to your issue.

1. Click the Create issue button.

1. In the issue metadata pane, click Edit next to the Labels field.

1. Select the Status::Open label, then click away from the metadata pane to apply the label to the issue.

1. Repeat the previous 2 steps to apply the Priority::Medium and Dev labels to the issue.

1. In the left pane, click Plan > Issues. You will see the issue you just created in the list along with its labels.

1. Create a second issue by clicking New issue in the top right of the issue list page.

1. In the Title section, type Backend services.

1. Paste the following in the Description section:

```
markdown
- Create DB
- Create service infrastructure
- Write documentation
```

1. Using the Assignees dropdown, assign the issue to yourself by clicking on the dropdown, and then clicking on your username.

1. Click the Create issue button.

1. Apply the following labels to the Backend services issue by clicking on tne label, then click away from the metadata pane to apply the label to the issue: Dev, Status::Open, and

Priority::High.

1. In the left pane, click Plan > Issues to see both issues with their labels.

1. Create a third issue by clicking New issue in the top right of the issue list page.

1. In the Title section, type Frontend services.

1. Paste the following in the Description section:

```
markdown
- UX design
- Integration
- Write documentation
```

1. Using the Assignees dropdown, assign the issue to yourself by clicking on the dropdown, and then clicking on your username.

1. Click Create issue.

1. Apply the following labels to the Frontend services by clicking on tne label, then click away from the metadata pane to apply the label to the issue: Dev, Status::WIP, and Priority::High.

1. In the left pane, click Plan > Issues to see all 3 issues with their labels.

Suggestions?

If you'd like to suggest changes, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab5.md

> Estimated time to complete: 45-60 minutes

Objectives

To help you manage your issues, GitLab provides metadata for each issue. Metadata allows you to define details for an issue such as weight, and due dates. In this lab, you will learn how to view and manage issue metadata.

You can learn more about issues and metadata in the documentation.

Task A. Set issue metadata

1. In your Family Budget Calculator project, click Plan > Issues from the left pane.

1. Click on the Third-party financial services integration issue that you created in a previous lab.

1. In the issue's metadata pane (on the right side of the screen), click Edit next to the Epic field.

1. Assign this issue to the Investment Tracking epic by clicking on the Investment Tracking in the list.

1. In the issue's metadata pane (on the right side of the screen), click Edit next to the Iteration field.

1. Assign this issue to the first most recent iteration in Team sprints by clicking on it in the list.

1. In the issue's metadata pane (on the right side of the screen), click Edit next to the Due date field.

1. Set the issue due date to 1 week from today's date by using the calendar.

1. In the issue's metadata pane (on the right side of the screen), click Edit next to the Labels field.

1. Apply the label Status::WIP. Note this replaces the previous Status::Open label, since an issue can't simultaneously have multiple labels with the same scope (the "Status::" part of the label).

> For more information about scoped labels, see the documentation.

1. In the issue's metadata pane, click Edit next to the Weight field.

1. Enter a value of 2, then click away from the metadata pane to set the issue's weight.

1. Navigate to Awesome Inc > Software group. In the left pane, click Epics. Click the Investment Tracking epic. Note that the Third-party financial services integration issue appears in the epic's details page.

1. In the left pane, click Plan > Iterations. Click the iteration to which you assigned the issue. Note that the Third-party financial services integration issue appears on the iteration's details page.

Task B. Organize, promote, and link issues and epics

1. In your Family Budget Calculator project, click Issues in the left pane.

1. Click your Backend services issue. You realize that, considering the scope of this initiative, this issue would be better suited as an epic, with each To-Do in the description as a separate issue in the epic.

1. To promote this issue to an epic, use the /promote quick action in the issue's comment

field, then click Comment.

> A quick action is a text-based shortcut for common actions that are usually done by selecting buttons or dropdowns in the GitLab user interface. You can enter these commands in the descriptions or comments of issues, epics, merge requests, and commits. For more information about quick actions, click here.
>
> You can also promote an issue to an epic by clicking the vertical ellipsis next to the Edit button, then clicking Promote to epic in the resulting menu.

1. After applying the quick action, Backend services is now an epic at the Core group level. Note that there is no comment that says '/promote', as quick actions do not leave a comment. Using the breadcrumbs at the top of the page, click your Core subgroup.

1. In the left pane, click Epics.

1. Click into your new Backend services epic.

1. You'd like to link the epic's To-Do items as individual issues. While you can promote issues into epics, you cannot promote To-Do items into an issues. Instead, you will need to create a new issue for each To-Do item. Under the Child issues and epics tab, click Add > Add a new issue.

1. Enter Create DB as the issue title.

1. In the Project dropdown, select Family Budget Calculator, as it is the only project created so far, and all issues must belong to a project.

1. Click Create issue in the top right corner.

1. Follow the previous four steps to create the Create service infrastructure and Backend documentation issues, both linked to the Backend services epic.

1. Click on the Create DB issue.

1. In the issue's right metadata pane, assign the issue to the Backend Services Deployed milestone.

1. Assign the Create service infrastructure and Backend documentation issues to the Backend Services Deployed milestone. So far all issues have been created in the Family Budget Calculator project by necessity. But as requirements grow it may be best to move some issues into more suitable projects.

1. Navigate to your Software > Core subgroup.

1. From the Core group landing page, click New project, then Create blank project.

1. In the Project name field, enter Database

1. Leave the Visibility Level at the default selection.

1. Enable the Initialize repository with a README checkbox.

1. Click Create project.

1. Return to your Family Budget Calculator project in the Core subgroup.

1. In the left pane, click Issues.

1. Click into the Create DB issue.

1. On the issue's details page, scroll to the bottom of the right metadata pane and click Move issue.

1. Select your Database project, then click Move.

1. See that the project heading and breadcrumbs at the top of the page indicate your issue is now part of the Database project.

Task C. Create and apply a description template

1. Navigate to your Awesome Inc group by using the breadcrumbs at the top of the page.

1. In the upper right corner of the group landing page, click New Project.

1. Click Create blank project.

1. Enter Description Templates in the Project name section. This project will hold templates that can be used to pre-populate issues and merge requests across the organization.

1. Enable the Initialize repository with a README checkbox, as we are not importing in a project that already exists from elsewhere. Leave all other options as their default.

1. Click Create project.

1. On the project landing page, click the (+) dropdown near the top of the page under the project title.

1. Click This Directory > New file.

1. In the File name field, enter .gitlab/issuetemplates/technicalspike.md

1. For the file's content, paste the following:

```
markdown
 Instructions
Use this issue to capture research that must take place before continued development of a feature.

 Summary
<!--In 2 sentences or fewer, describe the problem to be solved or the question to be
```

answered. -->

Impact Statement
<!-- Describe importance of solving the problem. How will it affect the feature or product direction? -->

Tasks

- [ ] Assign participants and DRI
- [ ] Apply appropriate priority and team labels
- [ ] Assign to an upcoming product sprint

/label ~"Status::Open"

> Notice that the template is using a quick action to assign a label. Whenever this template is used, a label will automatically be assigned when it is created.

1. Click Commit changes.

> Security Warning: Making direct commits on the main branch without a merge request is not recommended for enterprise level projects.

1. Using the breadcrumbs at the top of the page, navigate back to your Awesome Inc group.

1. In the left pane, click Settings > General.

1. Scroll to the Templates section and select Expand.

1. In the Templates dropdown, select your Description Templates project.

1. Click Save changes.

1. Now the template can be applied when creating an issue. Navigate to your Awesome Inc > Software > Core > Database project.

1. In the left pane, click Plan > Issues.

1. Click New issue.

1. In the Title field, type Identify tuning parameters to reduce performance bottlenecks

1. In the Description field, expand the Choose a template dropdown and select your technicalspike description template.

1. (Optional): Edit the description to fill in more details about the issue.

1. Assign the issue to yourself by clicking on the Assign to me link and click Create issue.

1. Review the pre-populated description and metadata on the issue's details page.

Suggestions?

If you'd like to suggest changes to the merge request, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab6.md

> Estimated time to complete: 45-60 minutes

Objectives

A merge request is a proposal to incorporate changes from a source branch to a target branch. Merge requests help you manage the changes that are applied to your code. In this lab, you will learn how to setup and manage merge request approval rules in your projects. You can learn more about merge requests in the documentation.

Approval rules define how many approvals a merge request must receive before it can be merged, and which users should do the approving. They can be used in conjunction with Code owners to ensure that changes are reviewed both by the group maintaining the feature, and any groups responsible for specific areas of oversight. See the documentation to learn more.

Task A. Set merge request approval rules

1. Navigate to your Database project inside the Software > Core subgroup.

1. In the left pane, click Settings > Merge Requests.

1. Scroll down to the Merge request approvals section and click Add approval rule.

1. In the Rule name field, enter Infra team

1. In the Groups field, select your Infrastructure group. You may need to use the 'All groups' option and search for your top-level group.

    > To help reduce the search results, try searching for /awesome/infrastructure, or yourgroupname/awesome/infrastructure.

1. Click Save changes.

1. Back in the Merge request approvals section of the Merge Requests page, click Add approval rule to create a second project-level rule.

1. In the Rule name field, enter Security operations

1. In the Groups field, select your Security group. You may need to use the 'all groups' option and search for your top-level group.

1. Click Save changes.

1. Back in the Merge request approvals section of the Merge Requests page, under Approval Settings, check the box next to Prevent editing approval rules in merge requests.

1. In the same section, ensure that Prevent approval by the author is unchecked. This is necessary so we can approve our own merge requests in the training environment.

1. Click Save changes.

Task B. Create a merge request

1. In your Database project, click Issues in the left pane.

1. Click into your Identify tuning parameters to reduce performance bottlenecks issue.

1. Click the down-arrow dropdown next to the Create merge request button on the issue landing page.

1. Ensure Create merge request and branch is checked.

1. In the Branch name field, change the text to read update-db-docs-perf-tools.

1. Ensure that the Source is set to main.

1. Click Create merge request.

1. Type Draft: Add performance tools to database documentation in the Title field.

> Putting 'Draft:' at the beginning of your merge request means that the merge request will not occur until it has been marked as ready. This is used to note that a merge request is not ready to be merged yet, and to prevent accidental merges. Note that the Mark as draft checkbox below the title will also check automatically when 'Draft:' is added to the title.

1. Remove Closes <ISSUE NUMBER> from the Description field. We want to keep the original issue open for additional work.

> If a merge request has 'Closes <ISSUE NUMBER>' in their description, the issue will be closed when the merge request is merged.

1. Verify that you are assigned to the merge request by checking the Assignees section. Also note any labels inherited from the issue, and any approval rules inherited from project settings.

1. Click the Create Merge Request button.

1. From the merge request details page, select Code > Open in Web IDE to edit files on the update-db-docs-perf-tools branch.

> The Web IDE is an advanced editor with built-in ability to commit to your repository branches. You can use the Web IDE to make changes to multiple files directly from the GitLab UI. See the documentation to learn more.

1. Click README.md from the left file pane.

1. Paste the following into README.md, beginning on line 3.

```
markdown
 Performance tools
The database currently uses HAProxy for load balancing.
We are researching and testing additional tools to improve performance.
```

1. Click Source Control in the left pane (the third button from the top).

1. In the Commit message field, enter Update docs with performance tools

1. Ensure that the message "Commit and push to update-db-docs-perf-tools branch" is written in the red 'commit' box underneath your commit message.

1. Click Commit and push to update-db-docs-perf-tools branch.

Task C. Perform code review and merge changes

> For advice on best practices when it comes to code reviews, see the documentation.

1. Navigate to the Draft: Add performance tools to database documentation merge request by clicking on the red GitLab button in the bottom left corner of the Web IDE, and then clicking Go to Database project on GitLab.

1. Click on Code > Merge Requests, and then click on Draft: Add performance tools to database documentation.

1. On the merge request page, click the Changes tab to see the changes that will be applied to the project's main branch after merge.

> Code reviewers can critique individual lines of code and suggest changes. See the documentation to learn more.

1. Click on the Overview tab.

1. In the three dots menu to the right of the issue title, click Mark as ready to take the merge request out of draft mode. You will see that Draft: has been removed from the merge request's title.

1. Click Approve to approve the merge request. Note that the Merge button now appears since all requisite approvals have been applied.

1. Scroll down to the comments field and type a comment in the comments field that reads: Approved. Ready to merge.. Click Comment to post the comment.

1. Click Merge and observe the merge complete successfully.

1. Navigate to the project landing page by clicking the Database title tile in the top left corner. See that the README.md file on the main branch now includes your updates.

1. In the left pane, click Code > Merge requests. The merge request will now appear under the Merged tab on this page.

Suggestions?

If you'd like to suggest changes, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab7.md

> Estimated time to complete: 30 minutes

Objectives

To help you plan, organize, and visualize a workflow, you can utilize an issue board.

The issue board is a software project management tool used to plan, organize, and visualize a workflow for a feature or product release. You can use it as a Kanban or a Scrum board. Issue boards can be configured to meet the needs of various project management frameworks.

You can learn more in the documentation.

In this lab, you will learn how to create a simple issue board.

Task A. View and customize project-level Issue boards

1. Navigate to your Family Budget Calculator project in the Software > Core subgroup.

2. In the left pane, click Plan > Issue boards.

> The default Development board contains only 2 lists by default: issues with the Open status and issues with the Closed status. For this lab, we are going to add more lists that use issue labels.

3. In the top right corner, click New list.

4. Ensure the Label radio button is selected in the Scope section of the list configuration pane. You can also create lists that are based on other metadata.

- Assignee: Who is directly responsible for an issue.

- Milestone: The release/due date of issues.

- Iteration: The velocity of issues.

5. Open the Value drop-down, and select Status::WIP.

6. Click Add to board. All issues tagged with Status::WIP should appear in the new list.

7. Add another custom list to the project board: in the top right corner, click New list.

8. The new list will be scoped by issue label. Verify that the Label radio button is selected in the Scope section of the list configuration pane.

9. Open the Value drop-down, and select Status::Done.

10. Click Add to board.

> Your Development board should now include the custom lists Status::WIP and Status::Done along with the default lists Open and Closed.

11. Note the Create service infrastructure issue in the Open list. Use your mouse to drag the Create service infrastructure issue into the Status::WIP list.

12. Click into the Create service infrastructure issue card by clicking directly on the issue's title. Note the Status::WIP label was automatically applied to that issue when you dragged it to a new list.

13. On the issue details page, click the Edit button next to Labels in the metadata pane.

14. Select the Status::Done label, and click away from the metadata pane to apply the label.

15. Return to Plan > Issue boards using the left pane.

> The Create service infrastructure issue should now appear in the Status::Done list.

Task B. Manage group-level issue boards

> Boards can also be viewed and managed at the group level.

1. Using the breadcrumbs at the top of the page, navigate to your top level Awesome Inc group.

1. In the left sidebar, navigate to Plan > Issue boards.

> Group-level boards will have a similar default board to project-level boards. This group-level issue board shows all issues across the group's subgroups and projects.

1. At the top of the page, click the options toggle directly to the right of the search bar. Toggle Epic swimlanes on. The board view should refresh and show a swimlane view of lists grouped by epic.

1. Scroll down to the bottom of the page and expand Issues with no epic assigned.

1. Click and drag the Identify tuning parameters to reduce performance bottlenecks issue into your Backend services epic.

1. Hover your mouse over the Backend services heading. Click the Go to epic link from the details box that appears.

1. Verify your Identify tuning parameters to reduce performance bottlenecks issue is part of the Backend services epic.

Task C. Create a new issue board

1. Navigate to Plan > Issue boards.

1. At the top of the page, click the Development dropdown to access the Switch board menu.

1. Click Create new board.

1. Title the board <YOUR NAME>

1. Deselect the checkboxes next to Show the Open list and Show the Closed list. This will remove the default lists from your custom board.

1. Click the Expand button next to Scope.

1. Click Edit next to Assignee and select yourself.

1. Click Create board.

1. In the top right corner, click New list.

1. Check that the Label radio button is selected in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Priority::High.

1. Click Add to board.

1. In the top right corner, click New list.

1. Check that the Label radio button is selected in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Status::WIP.

1. Click Add to board.

1. In the top right corner, click New list.

1. Click on the Milestone radio button in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Backend services deployed.

1. Click Add to board.

1. Refresh the browser tab that contains your new board.

Task D: Create a new issue for your board

1. In the Priority::High list, click the (+) icon to create a new high-priority issue.

1. Title the issue Update family budget app personas

1. Select Family Budget Calculator as the project for the issue to belong to.

1. Click Create issue.

1. An issue details pane should appear on the right side of the page. Assign the issue to yourself if it is not already assigned. Add an additional Status::Open label to the issue.

1. Click the X in the top right corner to close out of the issue details pane.

1. Click the diagonal arrows in the top right of the page to enter Focus mode. The rest of the GitLab navigation UI is now hidden, allowing you to focus on issues in the board.

1. Click the diagonal arrow again to leave Focus mode.

Suggestions?

If you'd like to suggest changes, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab8.md

> Estimated time to complete: 30 minutes

Objectives

Kanban boards show you the progress of all tickets in your project, as they move from the Backlog, to being worked on, to being checked by QA, to being closed. A real-world Kanban board might involve many more statuses than these 4, but these are adequate for a bare-bones Kanban demonstration. To learn more about Kanban boards, click here.

Task A. Create a new subgroup to work in

1. In the GitLab environment, navigate to the Awesome Inc group.

1. Click on the New subgroup button in the top right.

1. Type PM Workflows into the Subgroup name field.

1. Click Create subgroup to create the subgroup.

Task B. Create epics

1. From your new subgroup, in the left pane, click Epics. We will be creating three epics.

1. Click the New Epic button in the top right.

1. Type Frontend into the Title (required) field. We will be leaving all the other fields of the epic blank.

1. Click Create epic.

1. Click the New Epic button in the top right.

1. Type Backend into the Title (required) field. We will be leaving all the other fields of the epic blank.

1. Click Create epic.

1. Click the New Epic button in the top right.

1. Type QA into the Title (required) field. We will be leaving all the other fields of the epic blank.

1. Click Create epic.

Task C. Use a project template with issues

> GitLab has a variety of project templates you can use to help jump start your projects. The one we will be using is called Sample GitLab Project, which will come with predefined issues. You can see all of GitLab's project template here.

1. Click the PM Workflows title tile in the left pane to return to the subgroup landing page.

1. At the top of the page, click New project.

1. Click the Create from template option in the top right section.

1. Scroll to the bottom of the built-in templates. Next to Sample GitLab Project, click on Preview to see the project in a new tab. It is always important to preview a project before using it as a template.

1. After you have explored the template, return to your previous tab, and click Use template.

1. Type in Awesome Software the Title section. Leave all other options as their default values.

1. Click the Create project button. GitLab will now import a project containing many pre-generated issues and merge requests. When the import process has finished, you will be taken to the new project's landing page.

Task D. Assign some issues to epics

1. In the left pane, click Plan > Issues.

1. Click an issue of your choice.

1. Assign the issue to the Backend epic using the right metadata pane on the issue's details page.

1. Assign another issue of your choice to the Frontend epic using the right metadata pane on the issue's details page.

1. Assign another issue of your choice to the QA epic using the right metadata pane on the issues's details page.

Task E. Create labels representing Kanban stages

1. Using the breadcrumbs at the top of the page, return to your PM Workflows subgroup.

1. In the left pane, click Manage > Labels.

1. Click the New Label button in the top right.

1. Type in Status::Backlog into the title field.

1. Feel free to pick a color for the label and type in a description. When you are satisfied, click Create label.

1. Click the New Label button in the top right.

1. Type in Status::WIP into the title field.

1. Feel free to pick a color for the label and type in a description. When you are satisfied, click Create label.

1. Click the New Label button in the top right.

1. Type in Status::QA into the title field.

1. Feel free to pick a color for the label and type in a description. When you are satisfied, click Create label.

> You are not making a label to mark an issue as "done." Instead, you will close each issue when it is done. This lets GitLab register the issue as complete in burndown/burnup charts and on roadmaps.

1. Click the New Label button in the top right.

1. Type in Health::On Track into the title field.

1. Feel free to pick a color for the label and type in a description. When you are satisfied, click Create label.

1. Click the New Label button in the top right.

1. Type in Health::Needs Attention into the title field.

1. Feel free to pick a color for the label and type in a description. When you are satisfied, click Create label.

1. Click the New Label button in the top right.

1. Type in Health::At Risk into the title field.

1. Feel free to pick a color for the label and type in a description. When you are satisfied, click Create label.

Task F. Put all issues in the Kanban backlog

> None of the issues have been worked on yet, so you need apply the Status::Backlog label to all of them. Fortunately you can perform bulk edits on issues.

1. In the left pane, click Issues.

1. Click Bulk Edit.

1. Select all the issues by clicking the faint checkbox to the left of the search bar above the list of issues.

1. In the right pane, select Status::Backlog from the Labels dropdown.

1. At the top of the right pane, click Update all to apply the label to all selected issues.

Task G. Create the Kanban board

1. In the left pane, click Plan > Issue Boards.

1. At the top of the page, click the Development dropdown.

1. Select Create new board from the Switch board menu.

1. Type Kanban in the Title section.

1. Deselect the Show the Open list checkbox, as we will not need it for this board.

1. Keep the Show the Closed list checkbox selected.

1. Kanban boards generally show all your issues, so don't set a scope for the board.

1. Click Create board.

Task H. Add a list for each stage

1. At the top right of the page, click Create list.

1. In the New list options, select Label as the list scope and Status::Backlog as the value.

1. Click Add to board.

1. At the top right of the page, click Create list.

1. In the New list options, select Label as the list scope and Status::WIP as the value.

1. Click Add to board.

1. At the top right of the page, click Create list.

1. In the New list options, select Label as the list scope and Status::QA as the value.

1. Click Add to board.

1. Refresh the browser page to force the 3 new lists to appear in the same order that you created them.

Task I. Limit the allowed amount of work in progress

1. Click the gear icon at the top of the Status::WIP list to open list settings.

1. Next to Work in progress limit, click Edit button and set the value to 3

1. Click the X in the top right corner to exit out of the list settings.

Task J. Simulate work on issues

1. Practice dragging issues between different lists. For example, place more than 3 issues in the WIP list, and notice the results.

1. At the top of the page, click the Preferences icon > Epic swimlanes toggle on.

1. At the bottom of the page, expand Issues with no epic assigned to see the full list of issues in the Backlog list.

1. Practice dragging more issues, both to different lists and to different epics.

Suggestions?

If you'd like to suggest changes, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabpmhandsonlab9.md

> Estimated time to complete: 30 minutes

Objectives

In this lab you'll configure a Scrum board and populate it with the same issues you used for the Kanban board. A Scrum board is a board that is used to keep track of the progress in a sprint. To learn more about Scrum, click here.

Task A. Make Scrum-specific labels

> Compared to Kanban boards, Scrum boards normally require 2 extra labels to represent the project and sprint backlogs.

1. Navigate to the Awesome Software project.

1. In the left sidebar, click Manage > Labels

1. Click the New Label button in the top right.

1. Type in Status::Project Backlog into the title field.

1. Feel free to pick a color for the label and type in a description. When you are satisfied, click Create label.

1. Click the New Label button in the top right.

1. Type in Status::Sprint Backlog into the title field.

1. Feel free to pick a color for the label and type in a description. When you are satisfied, click Create label.

Task B. Put all issues in the Project Backlog

1. In the left pane, click Issues.

1. Using the bulk issue edit technique you learned in the last lab, apply the Status::Project Backlog label to all issues. Because this is a scoped label, it will remove any other Status labels that are already applied to any issues.

Task C. Make an iteration (sprint)

1. Iterations can only exist at the group or subgroup level. Using the breadcrumb trail, navigate to the PM Workflows subgroup.

1. In the left pane, click Plan > Iterations.

1. Click New iteration cadence.

1. Type Sprint 6 in the Title section.

1. Select today's date via the calender in the Start Date section.

1. Select 2 as the number of weeks in the Duration section.

1. Select 2 as the number of iterations in the Upcoming Iterations section.

> For the purposes of this lab, we will not worry about rollover, so there is no need to mark it.

1. Click Create cadence.

Task D. Add some issues to the next sprint

> This step simulates the "sprint planning" ceremony, where your team decides which issues it will work on in the upcoming sprint.

1. Navigate to the Awesome Software project.

1. In the left sidebar, click Issues.

1. Pick a handful of issues that you want to work on in the next sprint and assign them to the Sprint 6 first iteration. You can do this either with the bulk issue edit feature (selecting only the relevant issues), or by visiting each issue's details page.

1. Apply the Status::Sprint Backlog label to those same issues. You can do this either with the bulk issue edit feature (selecting only the relevant issues), or by visiting each issue's details page.

Task E. Make a Scrum board for the next sprint

1. In the left pane, click on Plan > Issue Boards and view your existing Kanban from the last lab.

1. Click Group by > Epic if the board is not already showing that view.

1. At the top of the page, click the dropdown to view the Switch boards menu.

1. Click Create new board.

1. Title the new board Sprint 6

1. Deselect the Show the Open list checkbox, as we do not need to have an Open list for this kind of board.

1. Leave the Show the Closed list checkbox selected.

1. Next to Scope, click Expand.

1. In the scope pane, click Edit near Iteration option. Select Current iteration.

1. Click Create board.

> We will now create three new lists for the board: a list with a Status::Sprint Backlog label, a list with a Status::WIP label, and a list with a Status::QA label.

1. In the top right corner, click Create list.

1. Check that the Label radio button is selected in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Status::Sprint Backlog.

1. Click Add to board.

1. In the top right corner, click Create list.

1. Check that the Label radio button is selected in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Health::Needs Attention.

1. Click Add to board.

1. In the top right corner, click Create list.

1. Check that the Label radio button is selected in the Scope section of the list configuration pane.

1. Open the Value drop-down, and select Status::QA.

1. Refresh the browser page to force the 3 new lists to appear in the same order that you created them.

> At this point, you have a complete board set up, but it is still recommended that you practice using this board as you would in production. For example, practice dragging and dropping your issues across different lists as their at-risk status increases or decreases. Or, simulate completing some issues by dragging them into the Closed list. Make sure to click on the issues themselves to confirm they are closed.

Task F. View your sprint progress

1. In the left pane, click Plan > Iterations.

1. Click into your Sprint 6 iteration, and then click on the first week in the list.

1. Note the progress of issues in the burndown and burnup charts.

Suggestions?

If you'd like to suggest changes, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabsecurityessentials.md

GitLab Security Essentials Lab Guides

Lab Name	Lab Link
Configure SAST and Secret Detection	Lab Link
Addressing Vulnerabilities	Lab Link
Dependency and IaC Scanning	Lab Link
Enable and Configure Container Scanning	Lab Link
DAST and API Scans	Lab Link
Enable and Scan Using a Scan Execution Policy	Lab Link

Quick links

Here are some quick links that may be useful when reviewing this Hands-On Guide.

GitLab Security Essentials Course Description
GitLab Security Essentials Certification Details)

Suggestions?

If you wish to make a change to the GitLab Security Essentials Hands-On Guide, please submit your changes via Merge Request!

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabsecurityessentialslab1.md

> Estimated time to complete: 15 minutes

Task A. Create a Project

For this section of the course, we will use a template with prepopulated code to allow us to test out our security scanners.

1. Select the New project button.

1. Select the Create from template tile.

1. Select the Instance tab.

1. Select Use template next to the Security Essentials Labs template.

1. In the Project name field, enter Security Labs.

> The project slug will automatically populate. You can change this to a shorter string if desired for your own project. Leave it at the default for this lab.

1. In the Project URL field, click the dropdown for the second half of the URL to make sure it's pointing to a group name (starts with training-users/) and not a username. You should create this project inside a group, not directly in your user's namespace.

1. Under Visibility Level, click Private.

> Since the parent group above your group is private, all child groups and projects below will be private. You can learn more about project visibility levels in the documentation.

1. Click Create project.

Task B. Enable and Configure SAST

> Static Application Security Testing, or SAST, is the process of examining source code for vulnerabilities. You can use a SAST scan to automatically scan a code repository for known vulnerabilities. You can also use a SAST scan to check merge requests for vulnerabilities before merging the request. This process helps ensure that your code stays vulnerability free.

1. Create a new file in the main branch by clicking (+) > This directory > New file.

1. In the Filename field, type .gitlab-ci.yml. You will see a template appear automatically that you can leave blank for this task.

1. Define a single test stage:

```yml
stages:
  - test
```

> Keep in mind that YAML files should be indented with two spaces. Your web IDE may try to use a tab with 4 spaces. Simply use the backspace to set 2 spaces if you are not copying and pasting the examples.

1. Enable SAST by pasting the following text after the stages definition in .gitlab-ci.yml:

```yml
include:
  - component: ilt.gitlabtraining.cloud/components/sast/sast@main
```

> It is also possible to configure SAST through the GitLab UI by navigating to Secure > Security configuration and clicking the Configure SAST button. We will be configuring it by editing the CI file for this lab to help you learn more about how it works under the hood.

1. Add an inputs section to the end of your .gitlab-ci.yml file and set the excludedpaths: venv/.

```yml
include:
  - component: ilt.gitlabtraining.cloud/components/sast/sast@main
    inputs:
      excludedpaths: venv/
```

> You can customize your SAST by adding configurations to the inputs section of the .gitlab-ci.yml file. For example, the excludedpaths variable can exclude project paths from the SAST scan. This option can be set to prevent unnecessary scanning of files.
>
> As an example, Python projects often contain a venv directory that contains packages used by the project. Since this directory does not contain our own source code, we should exclude it from the SAST scan.
>
> A full list of SAST variables can be found in the documentation.

1. Once complete, you will have a .gitlab-ci.yml file that looks like this:

```yml
stages:
  - test

include:
  - component: ilt.gitlabtraining.cloud/components/sast/sast@main
    inputs:
      excludedpaths: venv/
```

1. Click the Commit changes button, and add an appropriate commit message (e.g Add SAST template to .gitlab-ci.yml).

1. Select the Commit to a new branch option, and change the branch name to add-security. Ensure that Create a new merge request for this change is checked, then click the Commit changes button.

1. In the New merge request window, enter any name for your merge request. Select Create merge request.

Task C. Merge Request Vulnerability Report

> One of the main goals of security scanning is to prevent insecure code from making it into a repository. You can use the merge request vulnerability report to see all of the vulnerabilities that were detected in a single merge request. Note that this report will only show vulnerabilities that are newly introduced in the current merge request. If a vulnerability already exists in the repository, it will not show here, but will show in the project level vulnerability report.

1. After the pipeline completes, refresh the merge request screen.

1. You should now see a message stating Security Scanning detected 15 new potential vulnerabilities.

1. Select View all pipeline findings.

1. Review the vulnerabilities shown in the MR. For now, we will merge these vulnerabilities so we can demonstrate other security features. In most cases however, you would aim to resolve the issues here.

1. Return to your merge request.

1. Leave Delete source branch checked and click Merge.

Task D. Enable Advanced SAST

To provide more thorough scanning and vulnerability detection, we will opt to enable Advanced SAST in our project. To do this, we need to add another input to our SAST scan component.

1. In the Left sidebar, navigate to Code > Repository.

1. Select your .gitlab-ci.yml file

1. Select Edit > Edit in pipeline editor.

1. Below the excludedpaths input, add another input named runadvancedsast with a value of true. Once complete, your file will look like this:

```
yml
stages:
- test

include:
- component: ilt.gitlabtraining.cloud/components/sast/sast@main
  inputs:
    excludedpaths: venv/
    runadvancedsast: true
```

1. Set the branch name to sast-update. Ensure that Start a new merge request with this change is checked, and add dd an appropriate commit message (ex. Add Advanced SAST functionality). Click the Commit changes button.

1. In the MR page after this, provide an appropriate title (such as Enabled Advanced SAST in our pipeline), and click Create Merge Request.

1. In the left sidebar, select Build > Pipelines.

1. Select your currently running pipeline. Note that there is now a job titled gitlab-advanced-sast.

1. When the pipeline successfully completes, return to your MR, and click on the Merge button.

The GitLab Advanced SAST scanner will provide us more utility from our SAST scanner. We will see how the results look when we investigate our vulnerability report later in the lab.

Task E. Enable and Configure Secret Detection

> In the last section, you applied SAST to detect vulnerabilities in your source code. In addition to scanning code for vulnerabilities, GitLab can also scan your code for secrets like keys and API tokens. Adding secret detection to your code will prevent leaking sensitive data in your repositories.

The Secret Detection job belongs to the test stage by default. Since your .gitlab-ci.yml already defines that stage, you don't need to define it again.

1. In the Left sidebar, navigate to Code > Repository.

1. Click the .gitlab-ci.yml file. In the top right above the code, navigate to Edit > Edit in pipeline editor.

1. Enable Secret Detection by adding the component at the end of the existing include: section in .gitlab-ci.yml, below the component for SAST. This indent should be at the same level as the previous template.

```
yml
include:
- component: ilt.gitlabtraining.cloud/components/sast/sast@main
  inputs:
    excludedpaths: venv/
    runadvancedsast: true
- component: ilt.gitlabtraining.cloud/components/secret-detection/secret-detection@main
```

> It is also possible to configure Secret Detection through the GitLab UI by navigating to Secure > Security configuration and clicking the Configure Secret Detection button. We will be configuring it by editing the .gitlab-ci.yml file for this lab to help you learn more about how it works under the hood.

1. Configure the Secret Detection job to ignore the test directory by pasting this job definition below your component import.

```
yml
secretdetection:
  variables:
    SECRETDETECTIONEXCLUDEDPATHS: tests/
```

> To configure Secret Detection to use non-default behavior, you can override the secretdetection job definition and add variables inside it.
>

> A full list of Secret Detection variables can be found in the documentation.

1. Your .gitlab-ci.yml file will now look like this:

```
yml
stages:
    - test

include:
  - component: ilt.gitlabtraining.cloud/components/sast/sast@main
    inputs:
      excludedpaths: venv/
      runadvancedsast: true
  - component: ilt.gitlabtraining.cloud/components/secret-detection/secret-detection@main

secretdetection:
  variables:
    SECRETDETECTIONEXCLUDEDPATHS: tests/
```

1. Set the branch to main and select Commit changes.

> Note: It is considered best practice that all changes to a repository should be via an MR, and that MRs should be approved by other team members before merging. However, since this project only has one member (the person working on this lab), such a review is not possible. Therefore, it is common and accepted in small teams and demo environments to make direct commits to the main branch. In real-world scenarios, you should use merge requests.

1. Navigate to Build > Pipelines, review your jobs, and wait for all pipelines to complete.

Task G. View the Project Level Vulnerability Report

> Every time you merge code into the main branch, the security pipeline will run and generate a project level vulnerability report that shows all vulnerabilities in the latest commit to the default branch. Think of this as the baseline set of vulnerabilities that you'll compare to vulnerabilities on other branches.

1. Navigate to Secure > Vulnerability Report.

1. Looking at the Report Type column in the Development vulnerabilities table, you'll see a variety of vulnerability detections for each tool we enabled.

1. We want to see only the vulnerabilities found by the Advanced SAST scanner. We can filter by selecting the Scanner dropdown in th Search or filter vulnerabilities... bar and selecting GitLab Advanced SAST .

1. Select one of the Improper neutralization of special elements used in an SQL command ('SQL Injection') vulnerability. You will see two tabs here, Details and Code flow. The Details tab shows general details about the vulnerability. The Code flow tab is a special feature provided by the advanced SAST scanner. This shows how your vulnerability is reached in your code.

1. Review these different results. In the next lab, we will discuss how to triage and resolve these vulnerabilities.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabsecurityessentialslab2.md

> Estimated time to complete: 15 minutes

In the last lab, you introduced the SAST and Secret Detection scanners into your project. In this lab, we will explore methods to triage and resolve vulnerabilities.

Task A. Vulnerability Triage Process

1. Navigate to your Security Labs project.

1. In the left sidebar, select Secure > Vulnerability Report. To start your triage process, it is recommended to sort your vulnerabilities by severity, focusing on vulnerabilities that have not yet been triaged. This is the default setting.

1. In the security report, select Severity to change the sort order. Ensure that the arrow is pointing down so that severity is sorted from highest to lowest.

1. Select the severity vulnerability GitLab Personal Access Token. It should be at the top or close to the top.

1. Review the vulnerability. You will see that the finding is valid, as the main.py file contains a GitLab API token.

1. In the top right corner, click Edit Vulnerability, and then choose Change status. Set the status to Confirmed, and click Change status.

1. Scroll down to the bottom of the page, and select Create issue.

1. You will see that the issue automatically populates the vulnerability title and details. Review the issue details, then select Create issue.

1. Return to Secure > Vulnerability Report.

1. Select the first instance of the vulnerability Improper neutralization of special elements

used in a SQL command ('SQL Injection').

1. Select the Code flow tab.

1. Review the code flow to see how the vulnerability occurs. In the top right corner, click Edit Vulnerability, and then choose Change status. Set the status to Confirmed, and click Change status.

1. Select the Details tab.

1. Scroll down to the bottom of the page, and select Create issue.

1. Review the issue and select Create issue.

At this point, we've created two issues to address as security issues in our application. Let's review the process for fixing these vulnerabilities.

Task B. Fixing Vulnerabilities

1. Navigate to Plan > Issues.

1. Select the issue titled Investigate vulnerability: GitLab personal access token.

1. Select the code location: main.py:5.

1. Select Edit > Open in Web IDE.

1. Select the main.py file.

1. Delete the line of code:

```
python
app.config['SECRETKEY'] = 'glpat-Li5iWgSuUmDXNShPsozE'
```

> In a real scenario, you will also need to rotate this key. Deleting the line of code only removes it from the current code, but the secret may still be contained in the Git history and should be considered compromised.

1. In the left sidebar, select the Source Control icon.

1. Type an appropriate commit message (e.g. 'Removed API key), and click Create a new branch and commit.

1. Press Enter to take the default branch name.

1. Select Create MR in the bottom right of the screen.

1. Select Create merge request at the bottom of the page.

1. Wait for the pipeline in the merge request to complete, and refresh the page. Click on View all pipeline findings.

1. Review the findings. You should no longer see the GitLab Personal Token issue in the security list.

1. Return to the MR, and select Merge to merge the security updates.

1. Try out solving the SQL injection vulnerability on your own!

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabsecurityessentialslab3.md

> Estimated time to complete: 15 minutes

Task A. Add Dependencies and IaC

Our initial project has been built and we want to start on the deployment process. There are two areas we want to configure for our project. The first area is dependencies for our application. The second area is infrastructure for our application deployment. Let's set these up in our project. To add dependencies to your Python project, complete the following steps.

1. Navigate to your Security Labs project.

1. Open the requirements.txt file and observe the dependencies in it.

python

```
This file is autogenerated by pip-compile with Python 3.12
by the following command:

  pip-compile --output-file=requirements.txt requirements.in

requests==2.27.1
pyopenssl
httplib2
flask
flask-restful
twisted
```

```
serviceidentity
flask-cors
flask-httpauth
pytz
```

> Note that for pip, you are required to provide the pip-compile header.

For Infrastructure as Code, you will start by deploying an S3 bucket to your environment. To do this, you can set up Terraform files with infrastructure definitions. To do this:

1. Navigate to your project.

1. Select + > New file.

1. In the Filename, enter s3.tf.

1. Add the following contents to the file:

```
tf
resource "awss3bucketpublicaccessblock" "publicaccess" {
    bucket = awss3bucketdemobucket.id
    blockpublicacls = false
    blockpublicpolicy = false
}
```

1. Select Commit changes, and keep the Commit to the current main branch selected. Select Commit changes.

This project will also use Docker for deployments. To enable this, we will use the pre-existing Dockerfile.

1. Navigate to your project.

1. Select the Dockerfile file, and Select Edit > Edit single file.

1. Remove the previous content, and add the following lines instead:

```
Dockerfile
FROM python:3.4-alpine
ADD main.py .
```

1. Select Commit changes, and keep the Commit to the current main branch selected. Select Commit changes.

Task B. Add dependency scanning

Now that you have dependencies added to your project, you want to ensure that the dependencies

do not contain any security vulnerabilities. To validate this, you can add Dependency Scanning to your project.

1. Open your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. Add the following line to your include block:

```
yml
  - component: ilt.gitlabtraining.cloud/components/dependency-scanning/main@main
```

1. Write an appropriate commit message (ex. "Added Dependcy scanning to pipeline"), ensure that youa are committing to the main branch, and select Commit changes.

To view the progress of your new pipeline:

1. In the left sidebar, select Build > Pipelines.

1. Select your most recent pipeline. You should now see a job titled dependency-scanning. Once this pipeline completes, you will be able to view the results of the security scan:

1. In the left sidebar, select Secure > Vulnerability report.

1. In the Vulnerability report, filter for the Dependency Scanning tool by clicking on the search bar, clicking Report Type and then clicking Dependency Scanning.

1. Click on each vulnerability to review the findings. In the results, you will see various vulnerabilities in our version of the requests library. Let's fix these issues in our requirements.txt file.

1. When you select a vulnerability in the report, you will see a target version number to fix each issue. The first vulnerability recommends an upgrade to version 2.32.0 or above, the second vulnerability recommends an upgrade to version 2.31.0 or above. From this, we can determine that 2.32.0 will fix all our vulnerabilities. To set this version, edit your existing requirements.txt file. Update the requests import to:

```
python

 This file is autogenerated by pip-compile with Python 3.12
 by the following command:

    pip-compile --output-file=requirements.txt requirements.in

requests==2.32.0
...
```

1. Commit these changes and verify that the vulnerability is no longer detected.

1. In addition to scanning for vulnerabilities, dependency scanning also provides a list of dependencies for your project. To view this, in the left sidebar, select Secure > Dependency list.

1. Review the components and versions listed. Note that the license for each dependency is also listed here.

Task C. Add IaC scanning

To add infrastructure as code scanning to your project:

1. Open your .gitlab-ci.yml file.

1. Select Edit > Edit in pipeline editor.

1. In the include section, add the following template:

```
yml
  - template: Jobs/SAST-IaC.gitlab-ci.yml
```

1. Select Commit changes.

1. Observe the resulting pipeline and note that there is now a kics-iac-sast job.

1. Wait for the pipeline to complete.

1. Navigate to Secure > Vulnerability Reports.

1. Review the results of your IaC scan. We can filter by selecting the Scanner dropdown in th Search or filter vulnerabilities... bar and selecting kics. These results will be labeled as the SAST tool. Some examples include Missing User Instructions and S3 bucket allows public policy.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabsecurityessentialslab4.md

> Estimated time to complete: 15 to 20 minutes

Objectives

Many projects depend on using containers that might contain vulnerabilities in the container image.

In this lab, you will learn how to scan for vulnerabilities in your containers.

Task A. Build the Docker image

> In this section you will define a job that builds a Docker image. To build a Docker image with a CI/CD pipeline job, you must use a GitLab Runner that's configured to use a Docker executor.

1. Navigate to Code > Repository and edit .gitlab-ci.yml.

1. Define a build stage by pasting this in your .gitlab-ci.yml, at the top of the stages list, before the test stage. Make sure it has the same indentation as the existing test stage beneath it:

```
yml
stages:
- build
- test
```

1. Name your new job and assign it to the build stage by pasting the code below at the end of .gitlab-ci.yml:

```
yml
build-and-push-docker-image:
  stage: build
```

1. Your job must run on a Docker image that contains Docker tools. This approach is sometimes called "Docker in Docker" or "dind". You'll need to specify a version of the image that we've tested and know to work well for this task. Paste this underneath the build-and-push-docker-image job that you added in the previous step:

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
```

1. Your job also needs a second Docker image that enables the Docker in Docker workflow. Specify the second image with the services keyword, by pasting this into your job definition:

```
yml
build-and-push-docker-image:
  stage: build
```

```
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
```

1. It's helpful to define a variable to hold the full name and version of the Docker image you're creating, because you'll need to refer to that information more than once. You can assemble the name and version out of predefined variables that GitLab provides (remember that predefined variables generally start with CI). Paste this into your job definition:

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
    IMAGE: $CIREGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
```

1. If you set a variable telling Docker not to use TLS, you won't have to worry about setting up security certificates. Add the DOCKERTLSCERTDIR variable.

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
    IMAGE: $CIREGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
    DOCKERTLSCERTDIR: ""
```

1. Tell Docker to build a Docker image using the recipe in Dockerfile. Add the script: and docker-build lines.

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
    IMAGE: $CIREGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
    DOCKERTLSCERTDIR: ""
  script:
    - docker build --tag $IMAGE .
```

Task B. Push the Docker image to Project Container Registry

> Your job needs to log in to the project's container registry so it can push your image to it. You can log in using a username, password, and registry URL that are stored in predefined variables.

1. Add the docker login line to the bottom of the script section.

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
    IMAGE: $CIREGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
    DOCKERTLSCERTDIR: ""
  script:
    - docker build --tag $IMAGE .
    - docker login --username $CIREGISTRYUSER --password $CIREGISTRYPASSWORD $CIREGISTR
```

1. Your job can push the image with a single Docker command. Add the docker push line to the bottom of the script section.

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
    IMAGE: $CIREGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
    DOCKERTLSCERTDIR: ""
  script:
    - docker build --tag $IMAGE .
    - docker login --username $CIREGISTRYUSER --password $CIREGISTRYPASSWORD $CIREGISTR
    - docker push $IMAGE
```

1. Your completed job definition should look like this. Make any corrections necessary to the job definition in your .gitlab-ci.yml.

```
yml
build-and-push-docker-image:
  stage: build
  image: docker:20.10.17
  services:
    - docker:20.10.17-dind
  variables:
```

```
    IMAGE: $CIREGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
    DOCKERTLSCERTDIR: ""
  script:
    - docker build --tag $IMAGE .
    - docker login --username $CIREGISTRYUSER --password $CIREGISTRYPASSWORD $CIREGISTR
    - docker push $IMAGE
```

1. Commit the changes to the main branch with an appropriate commit message (Adding a Docker build job).

1. Navigate to Build > Pipelines to watch the progress of the new pipeline. Click on the pipeline to view the CI output for the build job.

1. When the pipeline finishes running, go to left navigation pane and click Deploy > Container Registry. Verify that your job created a new Docker image and pushed it into the project's container registry.

Task C. Enable Container Scanning

> Your application's docker image may contain known vulnerabilities. In order to prevent these vulnerabilities from reaching production, you can detect them with Container Scanning. Now that your Docker image is being built and pushed, you can enable Container Scanning.

1. Add the Container Scanning template to the existing include: section of .gitlab-ci.yml:

```
yml
- component: ilt.gitlabtraining.cloud/components/container-scanning/container-scanning@main
```

> This can be added anywhere in the list of templates.

1. We will need to make sure the Container Scanner is aware of the container that we want to scan, so to do so, we need to override the containerscanning job. Copy the code below to override the CSIMAGE variable for the containerscanning job:

```
yml
containerscanning:
  variables:
    CSIMAGE: $CIREGISTRYIMAGE/$CICOMMITREFSLUG:$CICOMMITSHA
```

1. Commit the changes with an appropriate commit message.

1. Navigate to Build > Pipelines to watch the progress of the new pipeline.

1. Open the containerscanning job to view the CI output. Wait for the pipeline to finish running.

Task D. View the results

1. Navigate to Secure > Vulnerability Report.

1. Select the Report Type filter, and select Container Scanning from the options.

1. The vulnerabilities listed are vulnerabilities detected inside of the Docker container you created. Click on any individual vulnerability to view more details.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab Security Essentials Hands-On Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/education-services/ilt-labs/gitlabsecurityessentialslab5.md

> Estimated time to complete: 40 minutes

Objectives

Many projects are deployed as web applications with API components. To be able to scan these components, you can utilize the DAST and API security scanners.

In this lab, you will learn how to implement both scanners for your projects.

Task A. Setting up DAST Scans

To test out DAST scans, we are going to set up an instance of a vulnerability web application called OWASP Juice Shop. Scanning this application will show you the full range of DAST scan results you can expect to see.

1. Create a new blank project. Name the project DAST.

1. In the DAST project, create a .gitlab-ci.yml file.

1. To start, add the DAST stage to your configuration:

```yml
stages:
  - dast
```

1. DAST currently uses a CI/CD template, which we will include just below our stages.

yml

```
include:
  - template: DAST.gitlab-ci.yml
```

1. Since we don't have a dedicated server, we will opt to pass the Juice Box application into DAST as a Docker service. To do this, start by defining the service below the template include:

```
yml
dast:
  services:
    - name: bkimminich/juice-shop:v16.0.0
      alias: juiceshop
```

1. We can provide many different variables to our DAST scanner. We will add the following values to the DAST scanner:

```
    yml
    variables:
        DASTTARGETURL: "http://juiceshop:3000/"
        DASTAUTHURL: "http://juiceshop:3000//login"
        DASTFULLSCAN: "false"
        DASTAUTHUSERNAME: "admin@juice-sh.op"
        DASTAUTHPASSWORD: "admin123"  use protected/masked variables, this is only for
demonstration purposes
        DASTAUTHUSERNAMEFIELD: "css:input[id=email]"
        DASTAUTHPASSWORDFIELD: "css:input[id=password]"
        DASTAUTHSUBMITFIELD: "css:button[id=loginButton]"
        DASTSCOPEEXCLUDEELEMENTS: "css:[id=navbar
```